

# COMP 3059 – Capstone Project I

## DriveTree

### Software Requirements Analysis and Design Assignment

#### Table of Contents

|                                                  |    |
|--------------------------------------------------|----|
| 1.0 Introduction.....                            | 2  |
| 1.1 Purpose.....                                 | 2  |
| 1.2 Scope.....                                   | 2  |
| 2.0 System Overview .....                        | 2  |
| 2.1 Project Perspective.....                     | 4  |
| 2.2 System Context.....                          | 4  |
| 2.3 General Constraints.....                     | 5  |
| 2.4 Assumptions and Dependencies .....           | 5  |
| 3.0 Functional Requirements .....                | 5  |
| 3.1 <Functional Requirement or Feature #1>.....  | 5  |
| 3.2 <i>Use Cases</i> .....                       | 6  |
| 3.2.1 Use Case #1 - Book Lesson.....             | 6  |
| 3.2.2 Use Case #2 - Instructor Verification..... | 7  |
| 3.2.3 Use Case #3 – Submit Review .....          | 8  |
| 3.3 <i>Data Modelling and Analysis</i> .....     | 9  |
| 4.0 Non-Functional Requirements .....            | 14 |
| 5.0 <i>Logical Database Requirements</i> .....   | 15 |
| 6.0 Other Requirements .....                     | 16 |
| 7.0 Approval .....                               | 17 |

## 1.0 Introduction

### 1.1 Purpose

The purpose of this document is to present a clear and comprehensive description of the DriveTree: Book · Learn · Go system

DriveTree is a web application designed to simplify how new drivers in Canada connect with verified driving instructors.

It outlines the major functional and non-functional requirements, the system context, and the overall design approach that the development team will follow.

This document acts as both a communication tool and a development guide for all project stakeholders—including students, instructors, and the admin—so that everyone understands what the application will accomplish and why it is being built.

By the end of Sprint 3, this document supports the transition from the planning phase to actual implementation and testing of the prototype

### 1.2 Scope

DriveTree's vision is to make learning to drive safer, easier, and more transparent for students and instructors alike.

In Canada, most learners still rely on phone calls, text messages, or social media to find lessons—processes that often lack proof of instructor credentials or any unified booking system.

DriveTree addresses this by offering a single online platform where students can search for instructors based on verified credentials, language, price, vehicle type, and location, then book and review lessons directly on the site.

Instructors benefit from a space to showcase their profiles, manage their schedules, and receive consistent bookings without paying for third-party ads.

Administrators maintain system trust by manually verifying licenses and moderating reviews.

This Sprint focuses on creating the foundation of that vision: the working backend APIs, the first functional React frontend, and clear design models that define how all modules communicate.

Future sprints will expand Drive Tree with payment integration, advanced analytics, and mobile compatibility.

## 2.0 System Overview

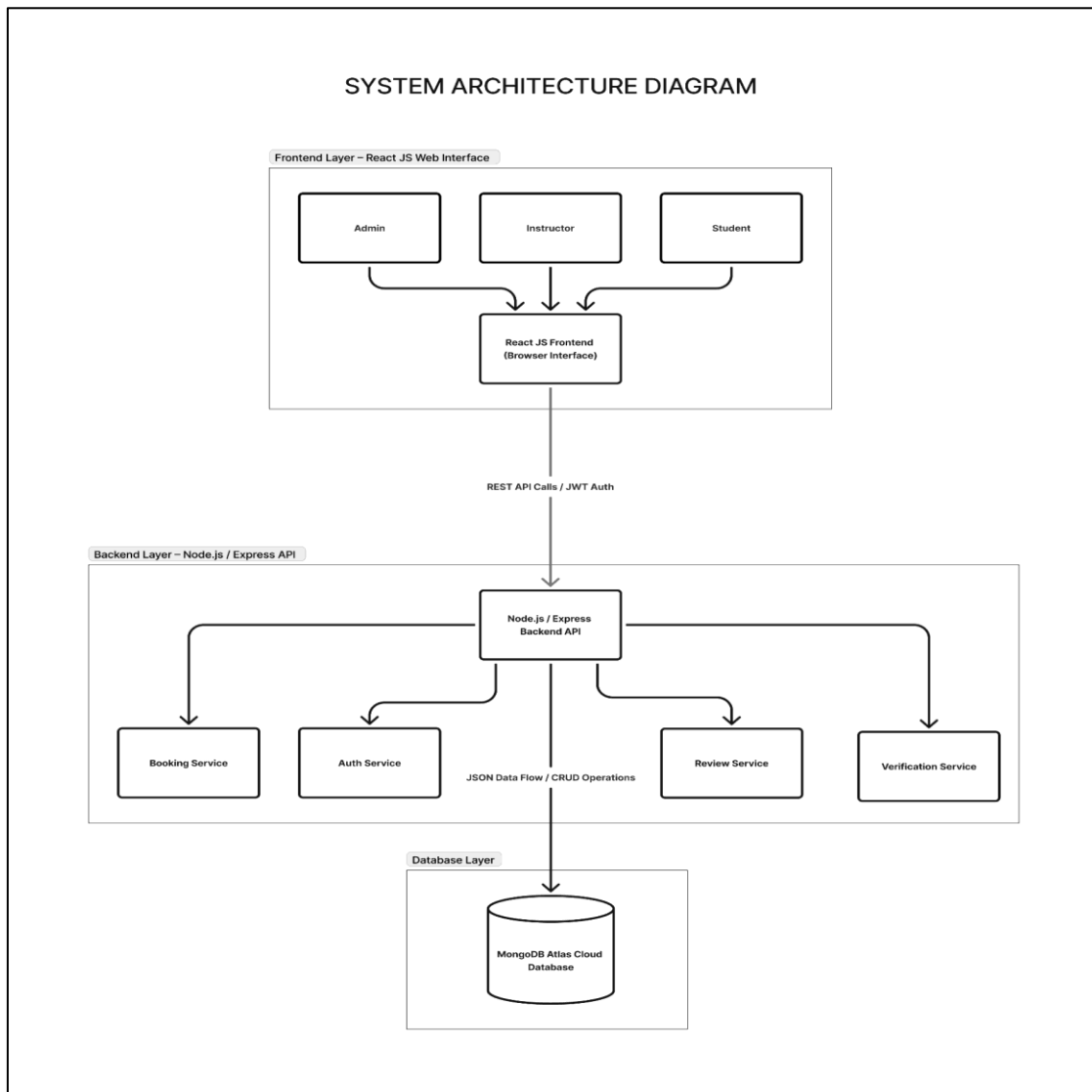
DriveTree operates as a **three-tier web application**:

1. **Frontend (Presentation Layer)** – Developed in React JS with Vite + Tailwind CSS.

It provides all user interfaces for login, registration, search, booking, and admin dashboards.

2. **Backend (Application Layer)** – Implemented using Node.js and Express. It exposes RESTful APIs to handle authentication, bookings, instructor verification, and review management. JSON Web Tokens (JWT) secure communication between client and server.
3. **Database (Data Layer)** – A MongoDB Atlas cloud instance stores users, bookings, reviews, and instructor data. Mongoose models maintain relationships between entities.

This structure ensures scalability, fast deployment, and easy maintenance.



## 2.1 Project Perspective

DriveTree is a new, self-contained system conceived by a team of George Brown College students as part of the Capstone Project I course.

It is not an upgrade or extension of any existing application.

Instead, it directly responds to a real problem: many students—especially newcomers to Canada—cannot easily confirm whether an instructor is licensed, affordable, or available in their area.

The team’s vision is to offer a modern digital solution that promotes trust and convenience.

DriveTree combines verification, scheduling, and review functionalities that currently exist separately across multiple sources.

The long-term perspective is to evolve the web version into a fully integrated ecosystem that includes mobile applications and payment automation.

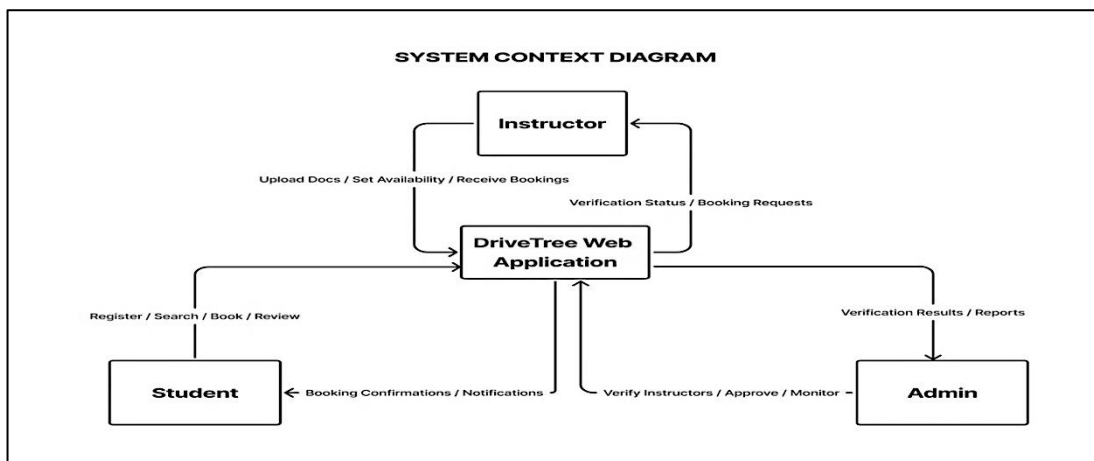
## 2.2 System Context

DriveTree involves three main actors:

- Student – Registers, searches instructors, books lessons, and submits reviews after each completed session.
- Instructor – Registers, uploads license documents for admin approval, sets available time slots, and receives bookings.
- Administrator – Reviews and approves instructor licenses, moderates feedback and ensures system reliability.

All users interact through the web interface, which communicates securely with the backend API.

Notifications such as verification updates or booking confirmations are sent through email.



## 2.3 General Constraints

- Development must follow the course timeline and grading criteria.
- Only free-tier hosting (Render, Vercel, MongoDB Atlas) is permitted.
- The MVP covers the **web platform only**—no native mobile apps yet.
- Instructor verification will remain manual in this sprint.
- Payment handling is postponed until future phases.
- Each team member must contribute code and documentation visible on GitHub.

## 2.4 Assumptions and Dependencies

- Users will access the system through a modern browser (Chrome, Edge, Firefox).
- Internet connectivity is required for all features.
- Instructors will provide legitimate, verifiable documentation.
- MongoDB Atlas and email-service availability are critical external dependencies.
- Future payments and AI-matching modules will depend on third-party APIs

## 3.0 Functional Requirements

### 3.1 <Functional Requirement or Feature #1>

- **FR-01 User Authentication**  
Users can create accounts, log in securely, and access features appropriate to their role.  
Passwords are hashed using bcrypt and sessions managed with JWT.
- **FR-02 Instructor Verification**  
Instructors upload their license images along with a real time photo; admins review them through a dashboard and assign statuses (Pending / Approved / Rejected).  
Only approved instructors appear in search results.
- **FR-03 Instructor Search and Profile**  
Students can filter instructors by city, hourly rate, language, and rating.  
Each result links to a detailed profile with photos, bio, and verified badge.
- **FR-04 Lesson Booking**  
Students select an available slot and confirm a lesson.  
The system checks conflicts and prevents double bookings.

- **FR-05 Review System**

After a lesson is marked as completed, students can rate their instructor and leave a written comment.

Reviews become visible after admin moderation.

- **FR-06 Admin Dashboard**

Admins manage all verification requests, review reports, and maintain platform data integrity.

- **FR-07 Learning Hub**

A static educational section with quizzes and tips to help students prepare for the written driving test.

### **3.2 Use Cases**

#### **3.2.1 Use Case #1 - Book Lesson**

**Actors:** Student, Instructor, System

**Goal:** Enable students to search for verified instructors and book driving lessons based on preferred time and location.

**Pre-conditions:** Student must be registered and logged in. Instructor must have an approved verification status and at least one available time slot.

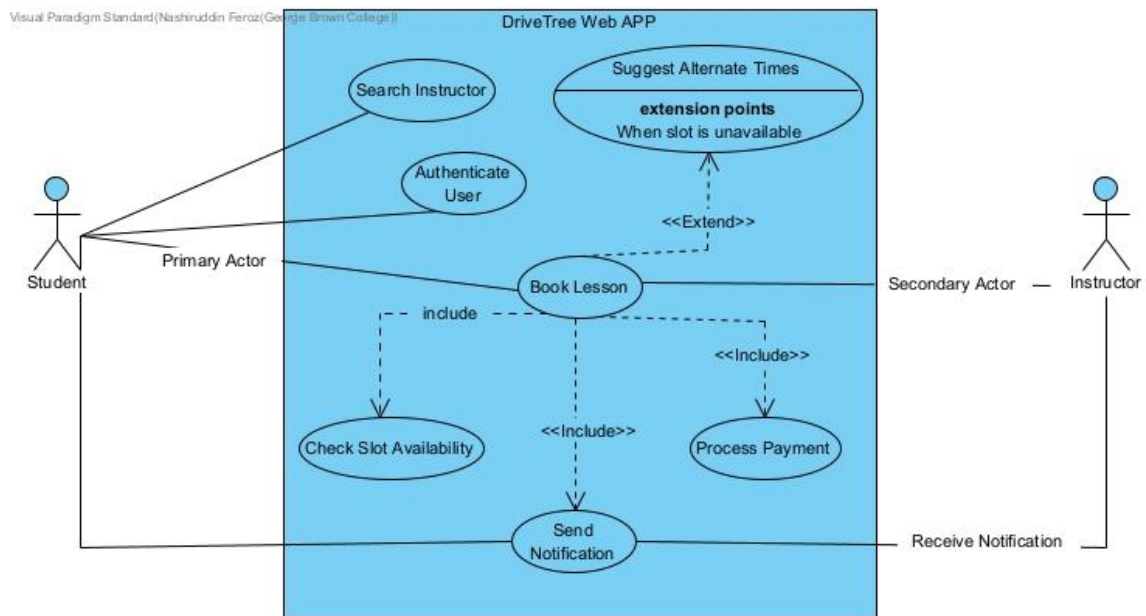
**Post-conditions:** Booking record created in the database; both Student and Instructor receive confirmation notifications.

**Main Flow:**

1. Student logs into the DriveTree web app.
2. Student searches for instructors by city, price, or language.
3. System returns a filtered list of verified instructors with availability.
4. Student selects an instructor and clicks "Book Lesson."
5. System checks if the selected slot is available.
6. If available, booking record is created and confirmation is displayed.
7. Email or dashboard notifications are sent to both parties.

**Alternative Flow:**

- If the selected slot is unavailable, the system shows "Time slot already booked" and suggests alternate times.
- If the instructor's account becomes unverified, booking is blocked.



### 3.2.2 Use Case #2 - Instructor Verification

**Actors:** Instructor, Admin

**Goal:** Allow an Admin to verify or reject instructor licenses to ensure only legitimate profiles appear in search results.

**Pre-conditions:** Instructor must be registered and logged in.

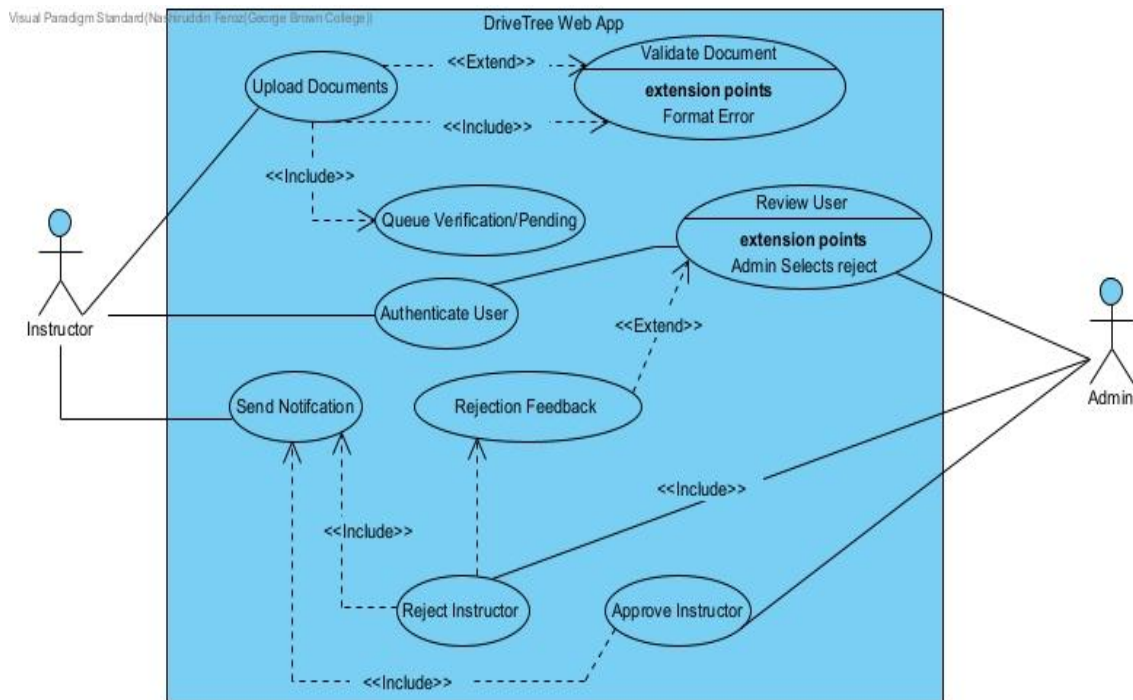
**Post-conditions:** Instructor's account marked as Verified/Rejected; status updated in database; notification sent to Instructor.

**Main Flow:**

1. Instructor logs in and navigates to Verification Page.
2. Instructor uploads license image (PDF/JPG/PNG).
3. System stores file and sets status = Pending.
4. Admin opens Dashboard → sees list of pending verifications.
5. Admin reviews document and selects Approve or Reject.
6. System updates instructor record and triggers email notification.

**Alternative Flow:**

- If document format invalid → validation error shown.
- If Admin rejects → Instructor receives feedback to reupload.



### 3.2.3 Use Case #3 – Submit Review

**Actors:** Student

**Goal:** Let students review and rate instructors after lesson completion.

**Pre-conditions:** Booking must be marked as Completed.

**Post-conditions:** Review stored in database; instructor's average rating recalculated and displayed.

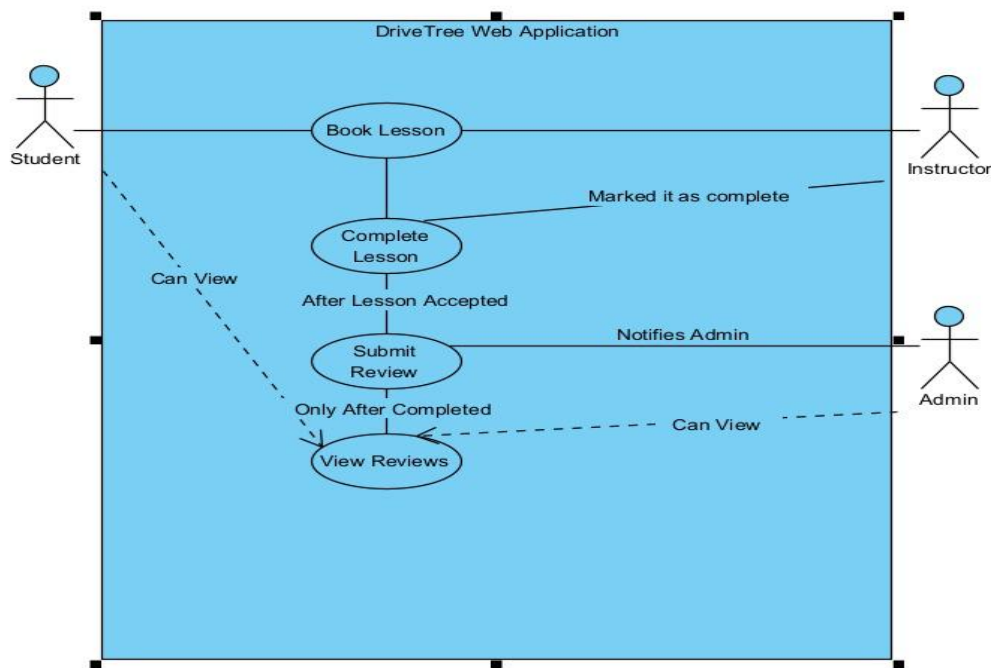
### Main Flow:

1. Student goes to “My Bookings.”
2. Selects a completed lesson and clicks “Write Review.”
3. Submits rating (1–5) and feedback comment.
4. System verifies lesson status = Completed.
5. Review stored; instructor’s profile updates automatically.

### Alternative Flow:

- If lesson not completed → “Review option unavailable.”

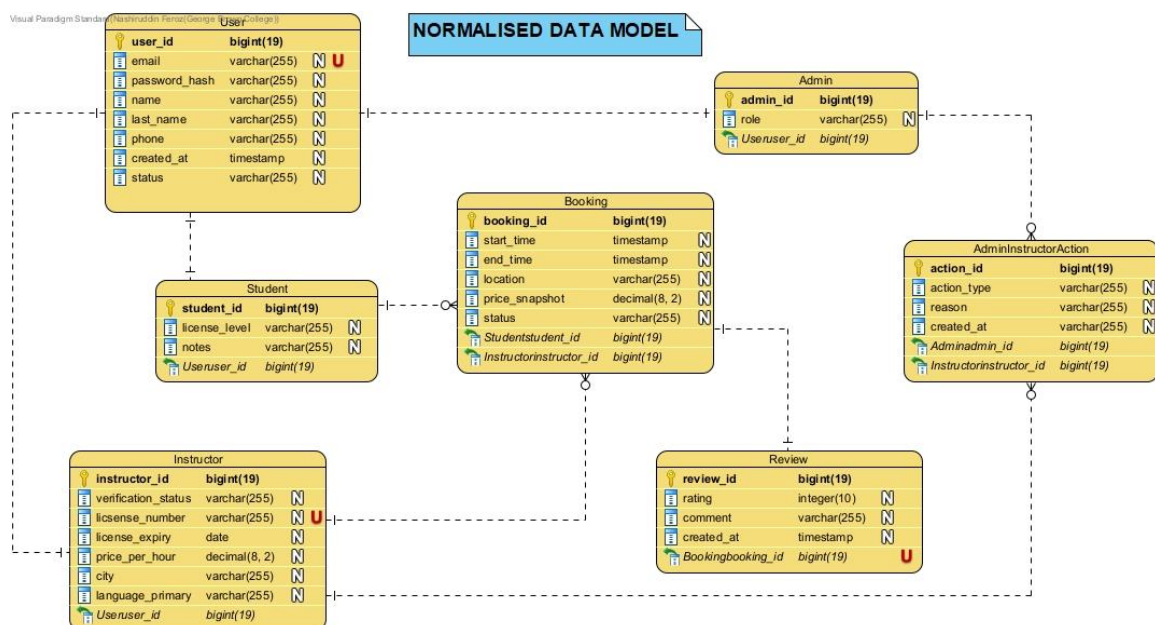


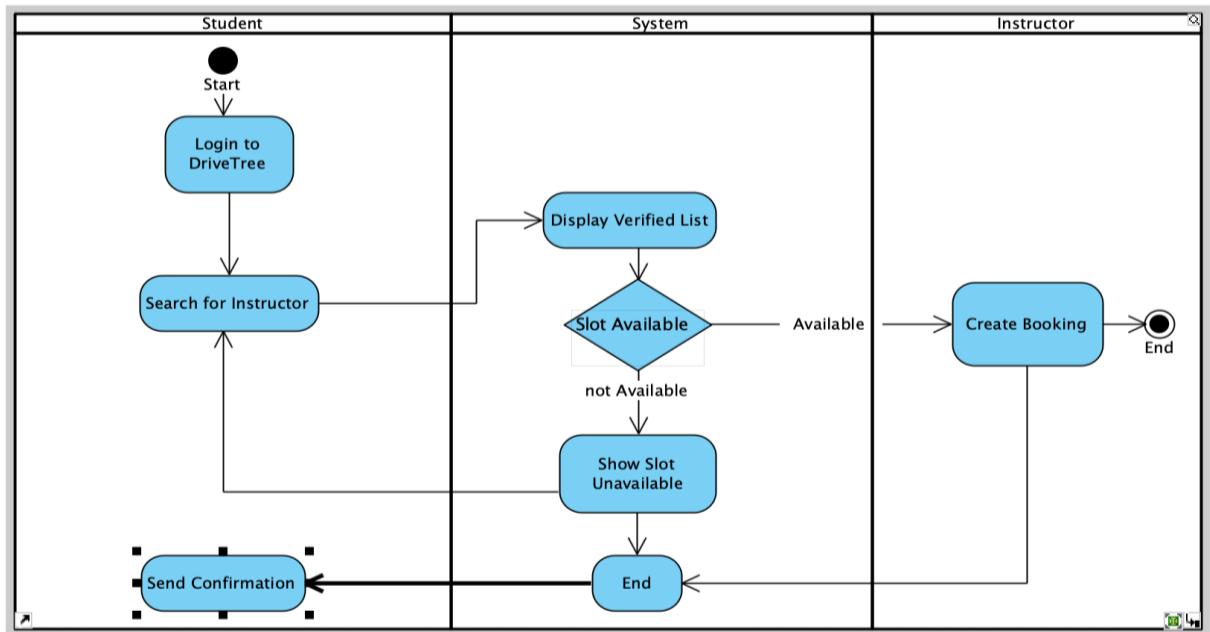
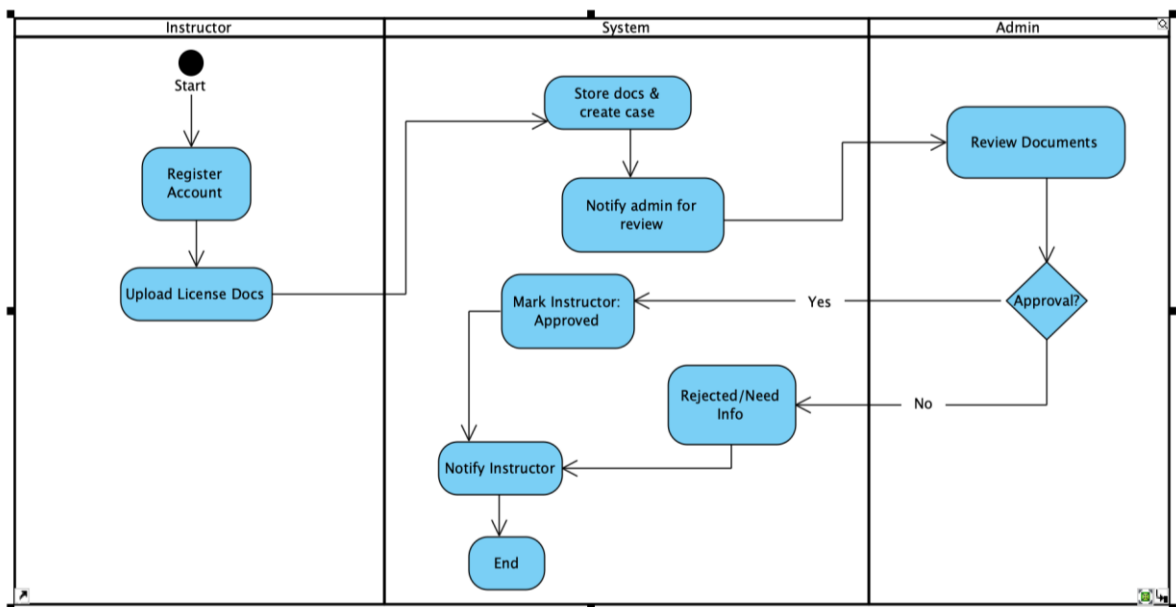


### 3.3 Data Modelling and Analysis

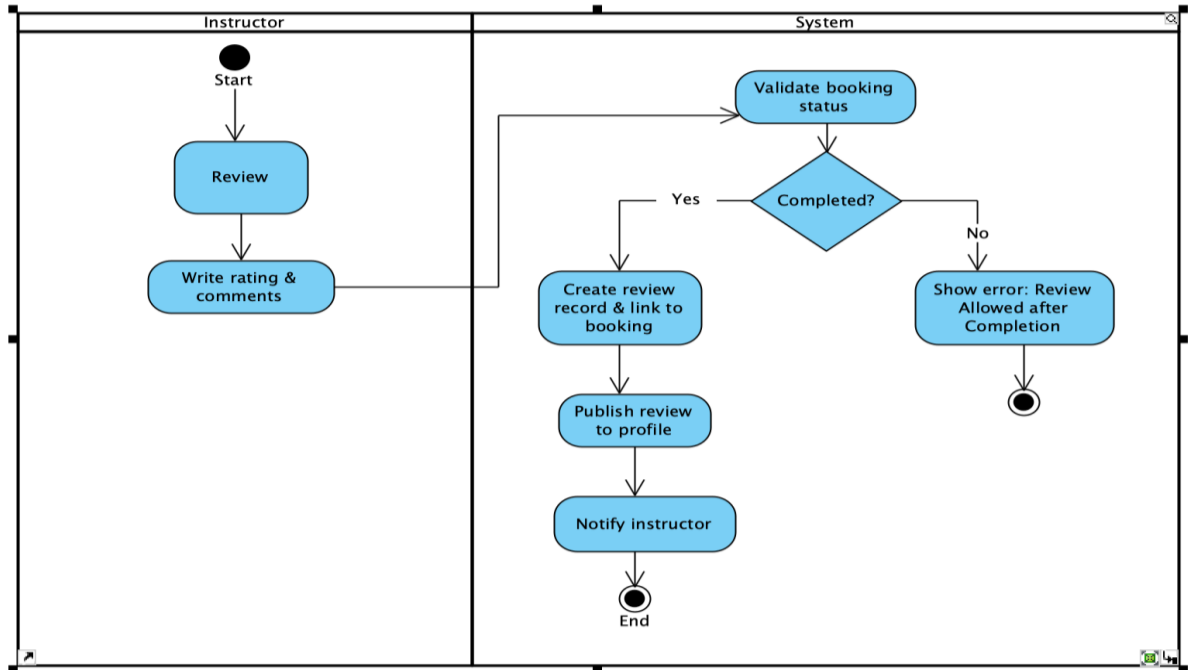
The database is designed around five main entities—Student, Instructor, Booking, Review, and Admin—linked through unique IDs in MongoDB.

- Normalized Data Model Diagram**

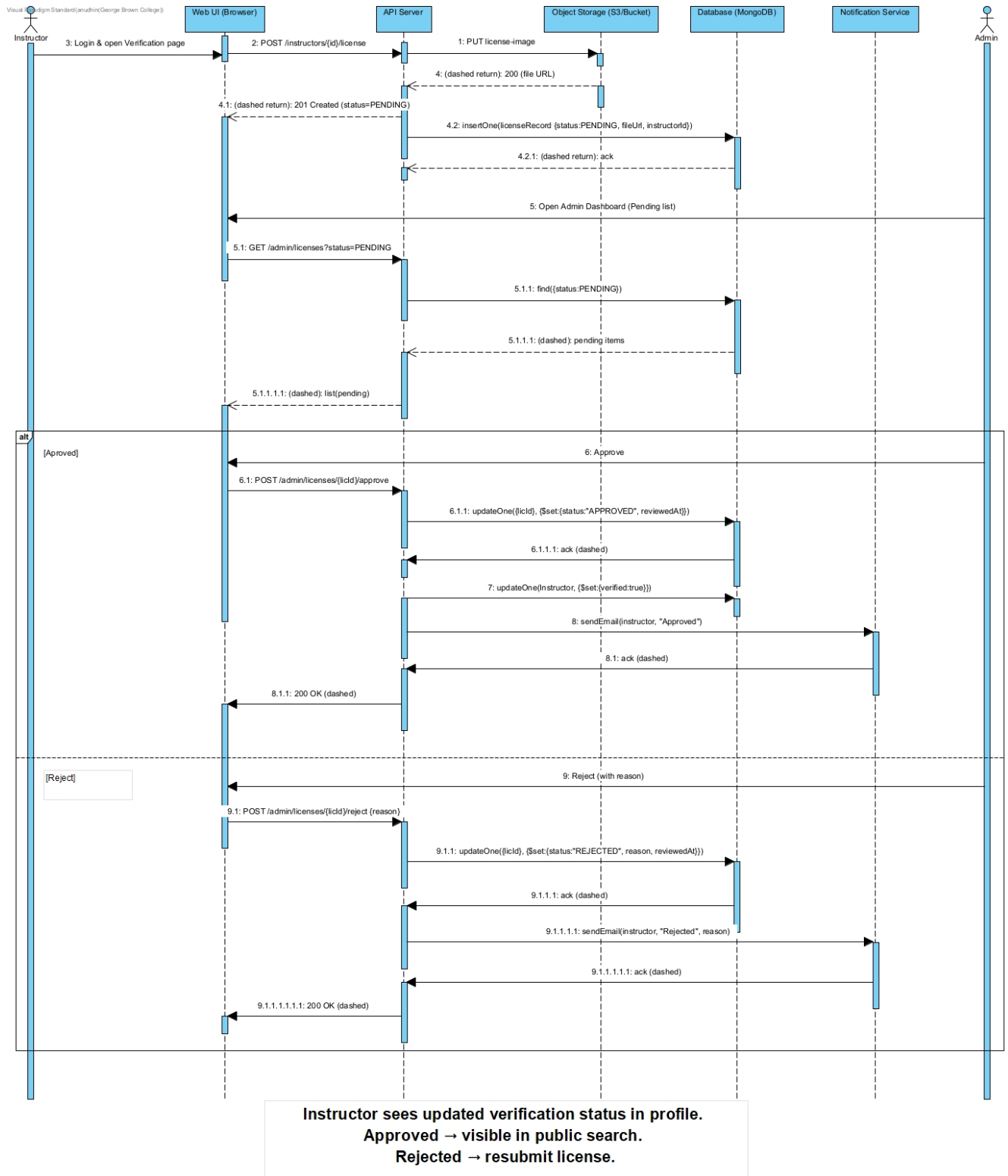


**Book Lesson Activity Diagram****Instructor Verification Activity**

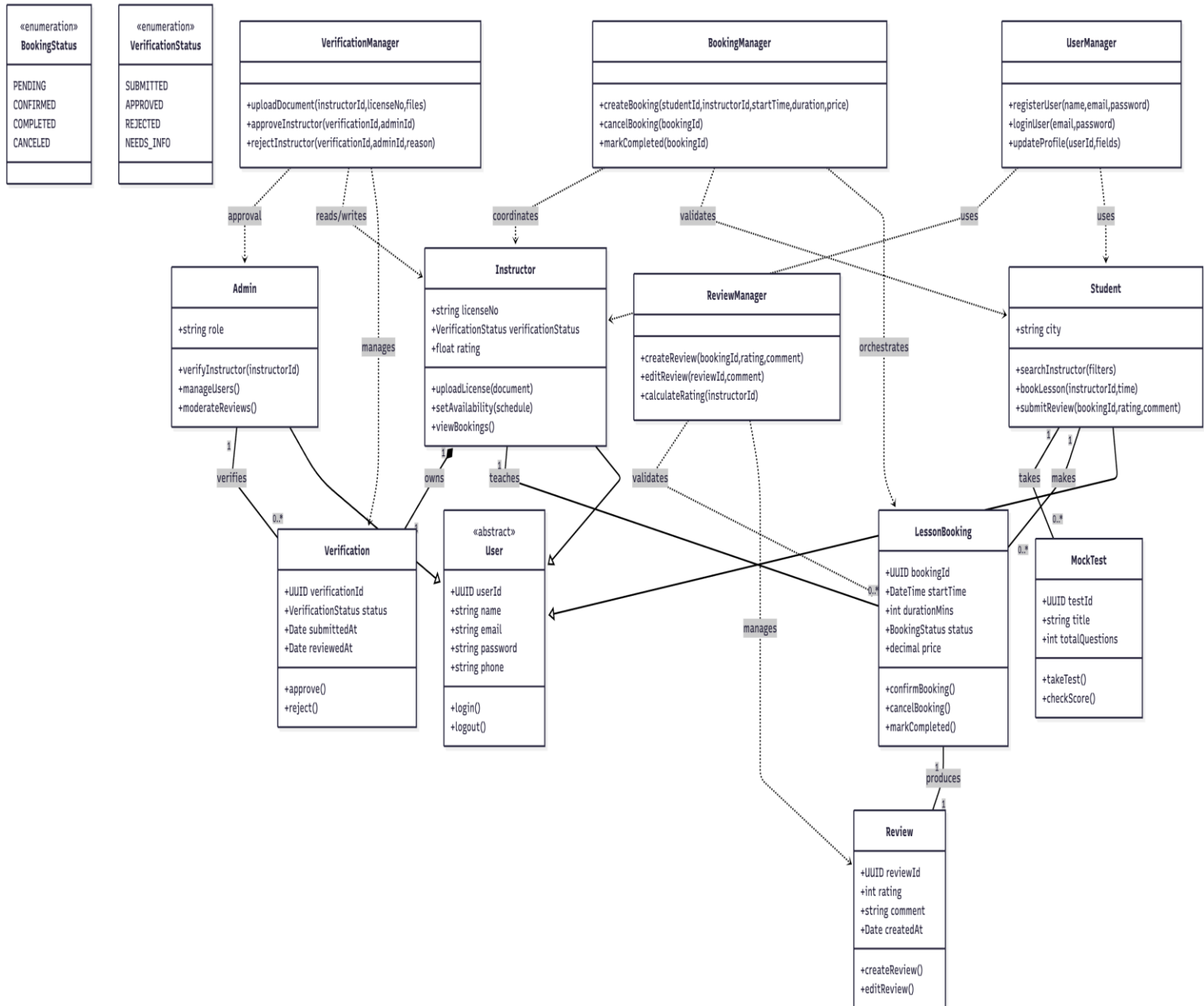
## Submit Review Activity



## Sequence Diagram



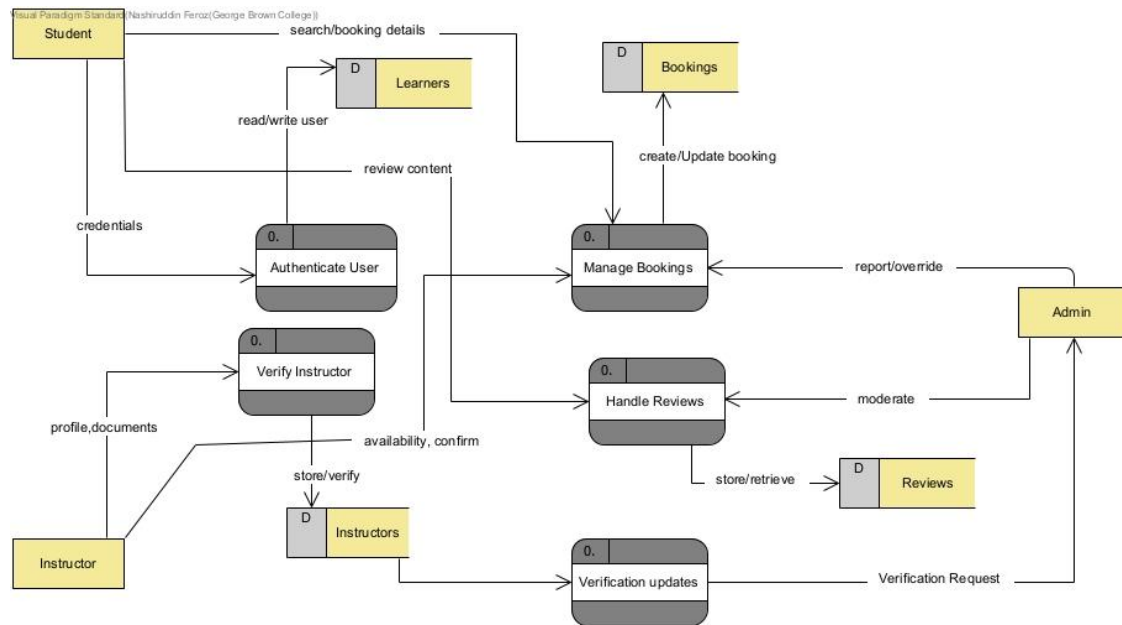
## • UML Class Diagram



### 3.4 Process Modelling

DriveTree's data flow includes authentication, booking, verification, and review processes.

Students and instructors act as external entities; the core system processes and updates data stores in MongoDB.



### 4.0 Non-Functional Requirements

| Category               | Detailed Requirement                                                                                                                                                                                                                                                |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Performance</b>     | 95 percent of API requests should complete within 2 seconds under normal load. Instructor search queries must return results within 1.5 seconds for up to five filters. The system should handle concurrent users efficiently without timeout or lag.               |
| <b>Reliability</b>     | System uptime must remain at or above 99 percent. Automated daily database backups are retained for 7 days. Critical operations such as booking and verification include fail-safe retries and error logging to prevent data loss.                                  |
| <b>Security</b>        | All communication occurs over HTTPS. User passwords are hashed with bcrypt and authenticated using JWT tokens. Role-based access (Student, Instructor, Admin) restricts sensitive features. File uploads are validated for type and size to prevent unsafe content. |
| <b>Usability</b>       | The web interface offers simple navigation, responsive layouts, and clear feedback messages. Form validation, confirmation prompts, and progress indicators improve the user experience for all roles.                                                              |
| <b>Maintainability</b> | The system follows a modular MVC structure with separate service and                                                                                                                                                                                                |

| Category      | Detailed Requirement                                                                                                                                                                                                 |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               | controller layers. Each module (Auth, Booking, Verification, Review) is well-commented and version-controlled through GitHub branches. Continuous Integration ensures early detection of issues.                     |
| Scalability   | The entire stack is containerized with Docker, allowing easy deployment on cloud environments. The backend can scale horizontally as user demand grows, supporting future conversion to a microservice architecture. |
| Accessibility | The application adheres to WCAG 2.1 AA standards, providing sufficient color contrast, visible focus states, and full keyboard navigation to ensure inclusivity for users with visual or mobility impairments        |

## 5.0 Logical Database Requirements

The DriveTree system uses a cloud-hosted MongoDB Atlas database to store all user, instructor, booking, and review information securely and efficiently. The logical database model follows a document-oriented structure with clear relationships between collections to support reliable and fast data retrieval.

### Database:

MongoDB Atlas is used as the main NoSQL database. It provides high availability, automatic scaling, and built-in backup management to ensure data persistence and minimal downtime.

### Collections:

1. **Users** – stores student and admin details such as name, email, password hash, and role.
2. **Instructors** – contains instructor profile data, license verification records, and available time slots.
3. **Bookings** – holds lesson details including student ID, instructor ID, date and time, and booking status.
4. **Reviews** – includes feedback and ratings linked to completed bookings.
5. **Availability** – keeps track of instructors' weekly schedules for lesson slot management.

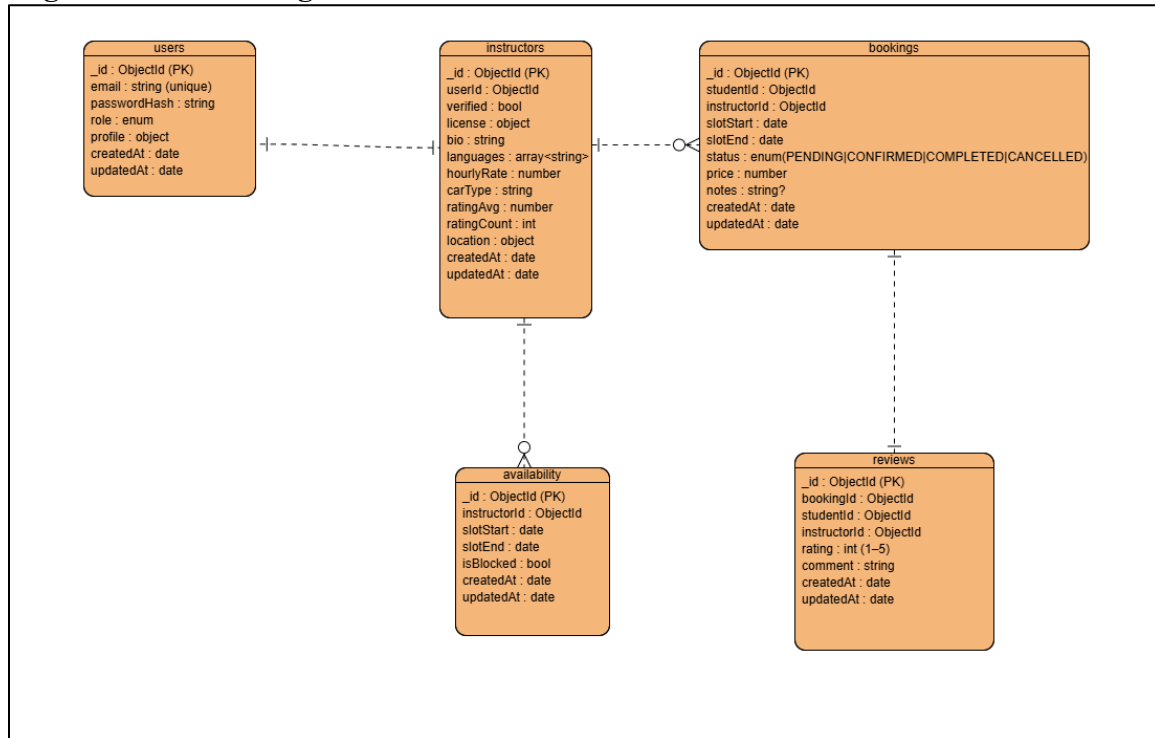
**Indexes:** A unique index is applied to the email field in the Users collection to prevent duplicate accounts. A compound index on instructorId and dateTime in the Bookings collection supports faster search and avoids overlapping lesson times.

**Data Integrity:** The system maintains referential consistency using ObjectId references between collections. Reviews can only be created for bookings with a status of "Completed" to ensure authenticity. Instructors that are not verified are excluded from student search results.

**Backup and Recovery:** MongoDB Atlas performs daily automated backups with a seven-day retention period. Backups are encrypted and stored securely in the same cloud region to enable quick recovery in case of data loss.

**Security and Access:** All database connections are made through a secure TLS/SSL channel. Only the backend API can access the database directly, with credentials stored in environment variables for safety.

### Logical Database Diagram



## 6.0 Other Requirements

### Frontend – React (Vite + Tailwind)

The user interface is built with React 18 and Vite for fast development and optimized builds. Tailwind CSS ensures a consistent, responsive design for Students, Instructors, and Admins.

Accessibility (WCAG 2.1 AA) and mobile compatibility are maintained throughout.

### Backend – Node 20 / Express 5

The backend uses Express 5 with modular routes for Auth, Instructor Verification, Booking, and Reviews. JWT authentication and bcrypt password hashing secure endpoints, while Mongoose schemas enforce data validation in MongoDB Atlas. Role-based access control and request validation protect API integrity.

### Deployment – Render (API) & Vercel (UI)

The API is hosted on Render and the web UI on Vercel. Both platforms provide



automatic build pipelines, HTTPS by default, and versioned deployments for rollback and staging environments.

### Version Control – GitHub Workflow

A shared GitHub repository is used with individual feature branches, Pull Requests, and peer review before merging into main. Commits follow a consistent naming convention to keep version history traceable.

### Testing – Postman & Jest

API endpoints are tested using Postman collections for functional coverage. Unit tests created with Jest verify controller and service logic. Error handling and response times are measured to ensure reliability.

### CI/CD – GitHub Actions

Automated pipelines run on every push: install → lint → test → build → deploy. Successful builds deploy the frontend to Vercel and backend to Render with status checks for each step.

### Observability & Security

Basic request logging (Morgan/Pino) and monitoring dashboards track uptime and latency.

Data is encrypted in transit (HTTPS) and at rest (MongoDB Atlas). Secrets are stored securely in environment variables, and uploads are validated for safe file types.

### Performance & Scalability

Target 95 % of API responses within 2 seconds; search queries under 1.5 seconds.

The system is containerized for easy scaling and future migration to a microservice model.

### Future Enhancements

Planned upgrades include Stripe payment integration, AI-based instructor matching, and an Android Compose mobile application that consumes the same REST API.

## 7.0 Approval

| Project Role | Name             | Signature | Date       |
|--------------|------------------|-----------|------------|
| Team Lead    | Heemal Syangbo   | H.S       | 2025-11-09 |
| Team Member  | Anudhin Thomas   | A.T       | 2025-11-09 |
| Team Member  | Jeffin Yohannan  | J.Y       | 2025-11-09 |
| Team Member  | Nasiruddin Feroz | N.F       | 2025-11-09 |